

FrontierAtlas / GraphOne — Technical Architecture

Production-minded MVP architecture for the AI Engineer Demo Task

1. Objective

Build a resilient ingestion platform for startups, products, research papers, AI jobs and AI news. The trial targets 1,000 startups, 1,000 products and 1,000 research papers, plus fresh jobs/news; the architecture is designed to scale to 500,000+ records by adding infrastructure rather than rewriting business logic.

2. Logical Architecture

Sources → URL Scheduler → Async Crawl Pool → Raw HTML/JSON → Content Cleaning + Date Normalization → Freshness/Idempotency → LLM Orchestrator → Schema Validation → Entity Resolution → SQL Store → Graph/Vector/Sheets.

3. Trial Source Set

News: TechCrunch AI, VentureBeat AI, The Verge AI, Hugging Face Blog, MarkTechPost. Jobs: RemoteFirstJobs AI, Jobicy, Himalayas, Remote OK, RemoteYeah. Startups: Y Combinator public directory. Products: Product Hunt GraphQL API. Research: arXiv API with optional GitHub enrichment.

4. LLM Orchestration

Use Gemini Flash → Groq Llama → DeepSeek as a fallback chain. The LLM is not the source of truth. Missing values remain null and source URLs are retained.

5. 413 Handling

Clean HTML, remove low-value elements, convert to text, split into semantic chunks, start with dense chunks and shrink the payload when a provider returns 413.

6. 429 Handling

Bound concurrency, retry 429/5xx with exponential backoff and jitter, then fall back to the next provider after provider-level exhaustion.

7. Freshness and Deduplication

Normalize timestamps and discard records older than 24 hours for jobs/news. Use URL-derived idempotency keys plus database uniqueness constraints so distributed workers cannot create duplicate records.

8. Entity Resolution

Normalize case, punctuation, whitespace and legal suffixes; use exact canonical lookup and conservative fuzzy matching. Log every raw-to-canonical decision.

9. Storage

Trial: SQLite. Production: PostgreSQL as the source of truth, object storage for raw pages, Redis for cache/locks, graph database for relationships and vector storage for semantic retrieval.

10. Scaling to 500k+

Move each stage behind durable queues. Run stateless crawler, LLM and persistence workers horizontally. Partition by source/entity/hash, autoscale replicas, add dead-letter queues, centralized metrics and tracing. Business logic remains unchanged.

11. Anti-Bot

Prefer official APIs and public feeds. Respect robots.txt, terms and rate limits. For permitted JavaScript-rendered sources use Playwright Async with bounded concurrency and caching. Do not attempt to defeat CAPTCHAs or security controls.

12. Observability

Track throughput, HTTP errors, 429/413 rates, retries, provider success, JSON validation, entity-match rate, duplicates, freshness lag, latency and dead-letter counts.

13. Integrity

Every record must trace to a legitimate source URL. Never fabricate missing values. In production, retain source content, extracted JSON, timestamps, provider/model and validation outcome for auditability.